

Busca Binária — Parte II

Universidade Estadual do Sudoeste da Bahia - UESB

2026.1

Motivação: Factory Machines

Enunciado: Uma fábrica possui n máquinas que podem ser usadas para fabricar produtos. Cada máquina leva k_i segundos para fabricar um único produto. As máquinas funcionam **simultaneamente**. Qual é o **menor tempo** necessário para fabricar um total de t produtos?

Entrada:

```
3 7
3 2 5
```

Saída:

```
8
```

upper_bound — Como Funciona?

Observe os 2 vetores ordenados abaixo. Vamos aplicar `upper_bound` do valor 3:

```
A: {1, 4, 5, 5, 8}  
B: {1, 2, 5, 5, 8}
```

Ao procurar um valor **maior que 3** nos arrays, há várias respostas possíveis:

- ▶ A: {4, 5, 5, 8}
- ▶ B: {5, 5, 8}

upper_bound — Como Funciona?

Observe os 2 vetores ordenados abaixo. Vamos aplicar `upper_bound` do valor 3:

```
A: {1, 4, 5, 5, 8}  
B: {1, 2, 5, 5, 8}
```

Ao procurar um valor **maior que 3** nos arrays, há várias respostas possíveis:

- ▶ A: {4, 5, 5, 8}
- ▶ B: {5, 5, 8}

Para que a busca binária encontre a **melhor resposta** (o valor imediatamente maior), deve-se armazenar as possíveis respostas à medida que o algoritmo executa. Há 2 formas:

- ▶ Mantê-la no intervalo de busca e retornar o último elemento resultante.
- ▶ Mantê-la em uma variável de resposta e atualizá-la à medida que melhores respostas forem encontradas.

Busca Simples vs. upper_bound

Busca simples (elemento exato):

```
int a = 0, b = n - 1;
while (a <= b) {
    int m = a + (b - a) / 2;
    if (array[m] == x) {
        return m;
    }
    if (array[m] > x)
        b = m - 1;
    else
        a = m + 1;
}
return -1;
```

upper_bound (primeiro maior que x):

```
int a = 0, b = n - 1;
int ans = -1;
while (a <= b) {
    int m = a + (b - a) / 2;
    if (array[m] > x) {
        ans = m;
        b = m - 1;
    } else {
        a = m + 1;
    }
}
```

Busca Binária na Resposta

E se fosse possível aplicar essa ideia de busca binária para encontrar a resposta ideal em outros problemas?

Busca Binária na Resposta

E se fosse possível aplicar essa ideia de busca binária para encontrar a resposta ideal em outros problemas?

Exemplo 1: suponha que você ficou responsável por decidir quantos servidores serão necessários para escalar horizontalmente a capacidade do sistema de sua empresa. Poucos serão insuficientes, porém muitos serão custosos.

Busca Binária na Resposta

E se fosse possível aplicar essa ideia de busca binária para encontrar a resposta ideal em outros problemas?

Exemplo 1: suponha que você ficou responsável por decidir quantos servidores serão necessários para escalar horizontalmente a capacidade do sistema de sua empresa. Poucos serão insuficientes, porém muitos serão custosos.

- ▶ Se você souber que m servidores são **insuficientes**, é necessário testar $m - 1$?
- ▶ Se você souber que m servidores são **suficientes**, é necessário testar $m + 1$?

Busca Binária na Resposta

E se fosse possível aplicar essa ideia de busca binária para encontrar a resposta ideal em outros problemas?

Exemplo 1: suponha que você ficou responsável por decidir quantos servidores serão necessários para escalar horizontalmente a capacidade do sistema de sua empresa. Poucos serão insuficientes, porém muitos serão custosos.

- ▶ Se você souber que m servidores são **insuficientes**, é necessário testar $m - 1$?
- ▶ Se você souber que m servidores são **suficientes**, é necessário testar $m + 1$?

Se a resposta correta for 12 servidores:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	...	20
F	F	F	F	F	F	F	F	F	F	F	T	T	T	...	T

O conjunto de testes é ordenado!

Busca Binária na Resposta — Exemplo 2

Determine se x é um quadrado perfeito.

Exemplo: $16 \rightarrow \text{SIM}$; $15 \rightarrow \text{NÃO}$

Busca Binária na Resposta — Exemplo 2

Determine se x é um quadrado perfeito.

Exemplo: $16 \rightarrow \text{SIM}$; $15 \rightarrow \text{NÃO}$

Todo quadrado perfeito é o quadrado de algum inteiro: $x = m^2$.

- ▶ Se $m^2 < x$, então $(m + 1)^2$ também será maior que x .
- ▶ Se $m^2 > x$, então $(m - 1)^2$ também será menor que x .

Busca Binária na Resposta — Exemplo 2

Determine se x é um quadrado perfeito.

Exemplo: $16 \rightarrow \text{SIM}$; $15 \rightarrow \text{NÃO}$

Todo quadrado perfeito é o quadrado de algum inteiro: $x = m^2$.

- ▶ Se $m^2 < x$, então $(m + 1)^2$ também será maior que x .
- ▶ Se $m^2 > x$, então $(m - 1)^2$ também será menor que x .

Se $x = 16$:

1	2	3	4	5	6	...	20
T	T	T	T	F	F	...	F

A imagem do conjunto de respostas também é ordenada!

Generalização da Busca Binária

- 1º A busca binária pode ser vista como um algoritmo responsável por **testar uma possível resposta**.

Generalização da Busca Binária

- 1º A busca binária pode ser vista como um algoritmo responsável por **testar uma possível resposta**.
- 2º Esse “teste” é uma função cujo parâmetro é a possível resposta e o resultado é **obrigatoriamente** True ou False.

Generalização da Busca Binária

- 1º A busca binária pode ser vista como um algoritmo responsável por **testar uma possível resposta**.
- 2º Esse “teste” é uma função cujo parâmetro é a possível resposta e o resultado é **obrigatoriamente** True ou False.
- 3º Para garantir que é possível inferir o comportamento de respostas maiores/menores, a função deve ser **monotônica** — ou seja, não decrescente ou não crescente.
Obs.: uma função monotônica é aquela que possui comportamento constante (apenas sobe ou apenas desce).

Generalização da Busca Binária

- 1º A busca binária pode ser vista como um algoritmo responsável por **testar uma possível resposta**.
- 2º Esse “teste” é uma função cujo parâmetro é a possível resposta e o resultado é **obrigatoriamente** True ou False.
- 3º Para garantir que é possível inferir o comportamento de respostas maiores/menores, a função deve ser **monotônica** — ou seja, não decrescente ou não crescente.
Obs.: uma função monotônica é aquela que possui comportamento constante (apenas sobe ou apenas desce).
- 4º Ao listar os valores do intervalo, sempre será possível associá-los a um prefixo/sufixo de True e False. A busca binária encontra o valor na **fronteira** entre ambos:
 - ▶ Encontrar o *lower bound* dos True → **Minimização**.
 - ▶ Encontrar o *upper bound* dos True → **Maximização**.

Implementação — Maximização e Minimização

Maximização (maior True):

```
int l = 0, r = n;
int ans = -1;
while (l <= r) {
    int m = l + (r - l) / 2;
    if (f(m)) {
        ans = m;
        l = m + 1;
    } else {
        r = m - 1;
    }
}
```

Minimização (menor True):

```
int l = 0, r = n;
int ans = -1;
while (l <= r) {
    int m = l + (r - l) / 2;
    if (f(m)) {
        ans = m;
        r = m - 1;
    } else {
        l = m + 1;
    }
}
```

Obs.: inicializa-se r com o maior valor possível de resposta e l com o menor.

Complexidade: $O(m \cdot \log n)$, onde m é a complexidade da função de teste e n é o intervalo de respostas.

Implementação — Exemplo 2 (Quadrado Perfeito)

```
int x;
bool f(long long m) {
    return m * m <= x;
}
int main() {
    cin >> x;
    long long l = 0, r = x, ans = -1;
    while (l <= r) {
        long long m = l + (r - l) / 2;
        if (f(m)) {
            ans = m;
            l = m + 1;
        } else {
            r = m - 1;
        }
    }
    if (ans * ans == x) cout << "YES" << '\n';
    else                cout << "NO"  << '\n';
}
```

Quando Usar Busca Binária na Resposta?

- 1º Ótimos indicadores são palavras-chave como “**Maximização**” e “**Minimização**”.
- 2º É possível testar se algum valor arbitrário x é uma possível resposta?
- 3º Se x é uma possível resposta, consigo garantir que todas as respostas **maiores/menores** também são?
- 4º Se x **não** é uma possível resposta, consigo garantir que todas as respostas maiores/menores também **não** são?

Atenção: cada problema possui uma abordagem distinta. **Não** utilize o mesmo código sempre — adapte a ideia ao que o problema propõe.

Motivação: Factory Machines — Revisão

Enunciado: Uma fábrica possui n máquinas que podem ser usadas para fabricar produtos. Cada máquina leva k_i segundos para fabricar um único produto. As máquinas funcionam **simultaneamente**, e você pode escolher livremente como distribuir o trabalho entre elas. Qual é o **menor tempo** necessário para fabricar um total de t produtos?

Entrada:

```
3 7
3 2 5
```

Saída:

```
8
```

Máquina 1: 2 produtos, Máquina 2: 4 produtos, Máquina 3: 1 produto.

Factory Machines — Solução (1/3)

1º — É possível testar se um valor arbitrário m é uma resposta?

Ou seja: é possível verificar se em m segundos as máquinas conseguem produzir t produtos?

Factory Machines — Solução (1/3)

1º — É possível testar se um valor arbitrário m é uma resposta?

Ou seja: é possível verificar se em m segundos as máquinas conseguem produzir t produtos?

Resposta: SIM. Para cada máquina i , ela produz $\lfloor m/k_i \rfloor$ produtos em m segundos. Basta somar a produção de todas as máquinas e verificar se a soma é $\geq t$.

Exemplo: máquinas com tempos 3 2 5. Em 5 segundos, conseguem produzir 7 produtos?

- ▶ Máquina 1: $\lfloor 5/3 \rfloor = 1$ produto
- ▶ Máquina 2: $\lfloor 5/2 \rfloor = 2$ produtos
- ▶ Máquina 3: $\lfloor 5/5 \rfloor = 1$ produto
- ▶ Total: $4 < 7$ — **não conseguiram.**

Factory Machines — Solução (2/3)

2º — Se 5 segundos não são suficientes, um tempo **menor** também não será?

Factory Machines — Solução (2/3)

2º — Se 5 segundos não são suficientes, um tempo **menor** também não será?

Resposta: SIM. Menos tempo implica menos produtos produzidos por cada máquina.

3º — Se em x segundos as máquinas conseguem produzir t produtos, em um tempo **maior** elas também vão conseguir?

Factory Machines — Solução (2/3)

2º — Se 5 segundos não são suficientes, um tempo **menor** também não será?

Resposta: SIM. Menos tempo implica menos produtos produzidos por cada máquina.

3º — Se em x segundos as máquinas conseguem produzir t produtos, em um tempo **maior** elas também vão conseguir?

Resposta: SIM. Mais tempo implica mais produtos produzidos.

Logo, a função é monotônica e é possível aplicar busca binária na resposta!

- ▶ Se o tempo for **suficiente**: guardar em uma variável de resposta e **diminuir** o intervalo de busca.
- ▶ Se o tempo for **insuficiente**: **aumentar** o intervalo de busca.

Hora de Implementar

Factory Machines

Implemente a solução utilizando busca binária na resposta.

Accepted

Tempo: 0.00s — Memória: 0 KB

Nos vemos na próxima aula.